Lecture 11: Defining sameness for complex data, part 1

Casting, type-testing, and “customized" type-testing mechanisms for checking sameness

Video Lessons

- Lesson 1 - Sameness of union data

Motivation

How can we test when two values are “the same”? This seemingly simple question has a surprisingly subtle answer, and we’ll spend the next two lectures constructing a reasonable answer.

> For now! As we’ll see in a few weeks, when we add new language features, we’ll have to revisit this and modify our approach.

As with any other operation, we need to determine the form of our code based on the structure of the data.

11.1 What do we mean by “sameness”?

We cannot define a new operation if we don’t know what we want it to do! Intuitively, we’d like “sameness” to mean “two values are the same if there’s no way we can tell them apart”. In other words, any method that we call that uses one value could use the other value and compute the exact same answer; there are no “trick questions” we could ask to distinguish the two values. In order to do that, and to make sameness be useful, there are a few other properties it must uphold:

- **Reflexivity:** every object should be the same as itself.
- **Symmetry:** if object `x` is the same as object `y`, then `y` is the same as `x`.
- **Transitivity:** if two objects are both the same as a third object, then they are the same as each other.
- **Totality:** we can compare any two objects of the same type, and obtain a correct answer.

As we define sameness for our various forms of data, keep these properties in mind, as they may indicate flaws in our designs that we must correct.

11.2 Review: sameness for built-in types

We can use Java’s built-in `==` operator to compare two integers, or two booleans:

```
t.checkExpect(4 == 4, true)
t.checkExpect(true == false, false)
```

Technically, we can use `==` to compare two doubles, but be careful! Doubles are imprecise, and two numbers that appear the same may not in fact be equal. It's always safer to subtract one double from the other, and compare the difference to a small value:

```
t.checkExpect(4.3333 == 4.3333, true) // BAD idea
t.checkExpect(4.3333 - 4.3333 < 0.001, true) // MUCH BETTER
```

(This latter one is conceptually what `checkInexact` does:

```
t.checkInexact(4.3333, 4.3333, 0.001);
```

But there is a difference between the two: the former line checks whether the *absolute* difference between two numbers is less than some tolerance. On the other hand, `checkInexact` looks at the *relative* difference. The distinction between these two is easier to see with larger numbers.

```
t.checkExpect(1100000 - 1000000 < 0.1, true); // FAILS: the absolute difference is much bigger than 0.1
t.checkInexact(1100000, 1000000, 0.1); // SUCCEEDS: the relative difference is at most 10%
```

Relative tolerances are interpreted as percentages of the expected value, while absolute tolerances are interpreted as plain numbers.)

To compare two strings, we use the `equals` method:

```
t.checkExpect("hello".equals("hel" + "lo"), true)
t.checkExpect("hello".equals("goodbye"), false)
```

Technically, we can use `==` to compare two strings, but it may not always give the answer we expect: it relies on subtle details of Java that we have not yet explored, and it is much safer to use the `equals` method.

> ***Do Now!***
>
> Convince yourself that `==` for integers and booleans and `equals` on `String`s obey the four properties above.

11.3 Review: sameness of structured data

Recall a simple definition of a book:

```
class Book {
  String title;
  String author;
  Book(String title, String author) {
    this.title = title;
    this.author = author;
  }
}
```

> ***Do Now!***
>
> How might we define sameness for books?

Two books are the same when their titles are the same and their authors are the same:

```
// In Book
boolean sameBook(Book that) {
  /* Fields:
   * this.title
   * this.author
   *
   * Methods of fields:
   * this.title.equals(String) -> Boolean
   * this.author.equals(String) -> Boolean
   *
   * Fields of parameters:
   * that.title
   * that.author
   */
  return this.title.equals(that.title) &&
         this.author.equals(that.author);
}
```

This makes sense: we define sameness of two structures in terms of the sameness of their parts. Also notice that we restrict our comparison to be between two `Book`s — it makes no sense to compare a `Book` to any other form of data, since they will never be the same.

> ***Do Now!***
>
> Define the method `samePoint` for the `CartPt` class.

> ***Do Now!***
>
> Revise the definition of `Book` so that its `author` field is now of type `Author`, where `Author`s have `first` and `last` names, and two `Author`s are the same when both names are the same. Revise the `sameBook` method. What methods must it delegate to?

11.4 Sameness of union data: Warmup

Recall a simplified form of our shapes examples:

```
interface IShape {
}
class Circle implements IShape {
  int x, y;
  int radius;
  Circle(int x, int y, int radius) {
    this.x = x;
    this.y = y;
    this.radius = radius;
  }
}
class Rect implements IShape {
  int x, y;
  int w, h;
  Rect(int x, int y, int w, int h) {
    this.x = x;
```

```
    this.y = y;
    this.w = w;
    this.h = h;
  }
}
```

We can define simple methods to check if two `Circle`s are the same, or two `Rect`s, by temporarily ignoring the fact that these classes implement `IShape`, and just defining methods as we did above with `Book`:

```
// In Circle
public boolean sameCircle(Circle that) {
  /* Template:
   * Fields:
   * this.x, this.y, this.radius
   *
   * Fields of parameters:
   * that.x, that.y, that.radius
   */
  return this.x == that.x &&
         this.y == that.y &&
         this.radius == that.radius;
}
// In Rect
public boolean sameRect(Rect that) {
  /* Template:
   * Fields:
   * this.x, this.y, this.w, this.h
   *
   * Fields of parameters:
   * that.x, that.y, that.w, that.h
   */
  return this.x == that.x &&
         this.y == that.y &&
         this.w == that.w &&
         this.h == that.h;
}
```

We can write tests for these methods, and they work correctly:

```
// In test method in an Examples class
Circle c1 = new Circle(3, 4, 5);
Circle c2 = new Circle(4, 5, 6);
Circle c3 = new Circle(3, 4, 5);
Rect r1 = new Rect(3, 4, 5, 5);
Rect r2 = new Rect(4, 5, 6, 7);
Rect r3 = new Rect(3, 4, 5, 5);

t.checkExpect(c1.sameCircle(c2), false)
t.checkExpect(c2.sameCircle(c1), false)
t.checkExpect(c1.sameCircle(c3), true)
t.checkExpect(c3.sameCircle(c1), true)

t.checkExpect(r1.sameRect(r2), false)
t.checkExpect(r2.sameRect(r1), false)
t.checkExpect(r1.sameRect(r3), true)
t.checkExpect(r3.sameRect(r1), true)
```

11.5 Sameness of union data: flawed attempt #1 using "casting" and type-testing

Ultimately the data type we care about here is `IShape`; we can't ignore it forever. We don't just want to compare `Circle`s to `Circle`s; we want to compare any `IShape` to any other `IShape`. If we follow the pattern we've started above, we ought to define a method

```
boolean sameShape(IShape that)
```

on the `IShape` interface, and implement it in our classes. Let's try it for `Circle` first:

```
// In Circle:
public boolean sameShape(IShape that) {
  /*
   * Fields:
   * this.x, this.y, this.radius
   *
   * Methods:
   * this.sameShape(IShape) -> boolean
   * this.sameCircle(Circle) -> boolean
   *
   * Methods on fields:
   *
   * Fields on parameters:
   *
   * Methods on parameters
   * that.sameShape(IShape) -> boolean
   */
   ???
}
```

Our template is exceedingly useless here: if all we know is that `that` is an `IShape`, we have no way of knowing whether `that` is a `Circle` or a `Rect` (or something else). So we can't call `sameCircle`, even though it would be perfect here.

11.5.1 Casting

One naive approach would be to try to somehow *force* Java to treat `that` as a `Circle` — after all, if `that` *isn't* a circle, then `that` can't possibly be equal to this circle. So why would any other situation be worth examining?

> ***Do Now!***
>
> What could go wrong with taking this position?

Java actually gives us a mechanism to achieve this "forcing": we can *cast* the value to the `Circle` class.

A *type cast*, or just a *cast*, is written as `((SomeTypeName)someValue)`, and it has two aspects to its meaning. First, it *statically* (that is, in the source code of our program) lets us treat `someValue` as if it had been defined with the type `SomeTypeName`, *even if we don't know for sure that its value is really of that type*, and so it will let us access the fields and methods of that type without giving a type error. This sounds great; let's try it:

```
// In Circle
```

```
public boolean sameShape(IShape that) {
  return this.sameCircle((Circle)that); // cast that to a Circle, and use the sameCircle helper
}
```

If we try writing tests for this, it doesn't go well:

```
// In the Examples class
t.checkExpect(c1.sameShape(c3), true)  // works
t.checkExpect(c1.sameShape(c2), false) // works
t.checkExpect(r1.sameShape(r3), true)  // works
t.checkExpect(r1.sameShape(r2), false) // works

t.checkExpect(c1.sameShape(r1), false) // CRASH! with a ClassCastException
```

> Quite literally, we're trying to fit a square peg into a round hole...

What went wrong? We passed `Circle`'s `sameShape` method a `Rect` value, and the code of the method tried to pretend that that `Rect` was actually a `Circle`, and that simply cannot work. So where did the exception get thrown? It came from the second part of the meaning of casts: *dynamically* (that is, at runtime), a cast will check that the given value really is an instance of the specified type. When it is, the cast succeeds and the program continues; but when it isn't, the cast will throw a `ClassCastException`.

> ***Do Now!***
>
> Which of our properties for sameness have we violated here?

11.5.2 Type-testing using `instanceof`

We've made some progress, but not enough. We can convince Java to compile our code without any type errors, but we can't safely use casts, since they may throw exceptions at runtime. When would casting a value to `Circle` fail? Precisely when the value is not a `Circle` (by definition) — and if the value is not a `Circle`, it definitely is not the same as this `Circle`. If we had some way to detect when a cast would fail, we could just return `false` and avoid using the cast at all.

In Racket, we had type-testing predicates (`circle?` and `rect?`) that would let us check what kind of a structure we had. Until now, we have said that Java has no such mechanism. In fact, Java actually does have a type-testing mechanism, but it does not work as well as we might hope. Let's see how to use it, and how it breaks.

To test whether a value is an instance of a given class or interface, we introduce a new operator, `instanceof`. We'll illustrate how it works by way of examples:

```
t.checkExpect(new Circle(3, 4, 5) instanceof Circle, true) // because Circle is a Circle.
t.checkExpect(new Circle(3, 4, 5) instanceof IShape, true) // because Circle implements IShape
t.checkExpect(new Circle(3, 4, 5) instanceof Book, false)  // because a Circle is not a Book
t.checkExpect(new Rect(3, 4, 5, 6) instanceof Circle, false) // because a Rect is not a Circle
```

Using `instanceof`, we can test a *value* against a class or interface *type*, and get a boolean answer. (This is unlike any other operator we have seen so far, which all let us work with two values, not with values and types.) Notice that we can ask absurd questions, like whether a `new Circle` value is an instanceof `Book`, even when we know the answer must always be `false`.

Let's now use this operator to improve our code above:

```
// In Circle
public boolean sameShape(IShape that) {
  if (that instanceof Circle) {
    // that is-a Circle -- we can safely cast!
    return this.sameCircle((Circle)that);
  }
  else {
    // that is not a Circle
    return false;
  }
}
```

Now our code works: when `that` is not a `Circle`, we just return `false`. When it *is* a `Circle`, we can safely cast it to `Circle` and then invoke `sameCircle` with it. Notice that we need both the cast and the `instanceof`: the cast convinces Java to let us use `that` as a `Circle`, and the `instanceof` test ensures that it actually will be safe for us to do so.

> This is a good example of the difference between *static* and *dynamic* information available in our code, and goes to show that static types sometimes *cause* problems (that must then be worked around), even as they help *solve* others.

If we try our tests, they work now:

```
// In the Examples class
t.checkExpect(c1.sameShape(r1), false) // works
```

> ***Do Now!***
>
> Implement `sameShape` for the `Rect` class, following the same pattern as for `Circle`. What test should you write to confirm that it works?

```
// In Rect
public boolean sameShape(IShape that) {
  if (that instanceof Rect) {
    // that is-a Rect -- we can safely cast!
    return this.sameRect((Rect)that);
  }
  else {
    // that is not a Rect
    return false;
  }
}
```

```
// In the Examples class
t.checkExpect(r1.sameShape(c1), false); // works
```

11.5.3 What goes wrong with casting and `instanceof`?

Let's add another type of shape to our classes: let's bring back the `Square` class as a subclass of `Rect`.

> ***Do Now!***
>
> Implement `sameShape` for `Square`, along with a `sameSquare` helper method.

> Write tests to confirm that it works properly.

We can start the definition of `Square` easily enough:

```
class Square extends Rect {
  Square(int x, int y, int s) { super(x, y, s, s); }
  public boolean sameShape(IShape that) {
    if (that instanceof Square) {
      return this.sameSquare((Square)that);
    }
    else {
      return false;
    }
  }
  public boolean sameSquare(Square that) {
    return this.x == that.x &&
           this.y == that.y &&
           this.w == that.w; // No need to check the h field, too...
  }
}
```

But writing tests for this shows a subtle bug:

```
// In test method in an Examples class
Square s1 = new Square(3, 4, 5);
Square s2 = new Square(4, 5, 6);
Square s3 = new Square(3, 4, 5);

// basic checks comparing two Squares should work
t.checkExpect(s1.sameShape(s2), false)
t.checkExpect(s2.sameShape(s1), false)
t.checkExpect(s1.sameShape(s3), true)
t.checkExpect(s3.sameShape(s1), true)

// Comparing a Square with a Rect of a different size
t.checkExpect(s1.sameShape(r2), false) // Good
t.checkExpect(r2.sameShape(s1), false) // Good
// Comparing a Square with a Rect of the same size
t.checkExpect(s1.sameShape(r1), false) // Good
t.checkExpect(r1.sameShape(s1), true)  // Not so good
```

> ***Do Now!***
>
> Which property of sameness have we violated?

11.6 Sameness of union data: flawed attempt #2 using "custom" type-testing

What's going on here? Carefully step through the last two examples. When we test `s1.sameShape(r1)`:

- `s1` is a `Square`, so we invoke the `sameShape` method defined in `Square`.
- This method checks whether `that` (which is `r1`) is an instance of `Square`, which it is *not*, so the method returns `false`.

On the other hand, when we test `r1.sameShape(s1)`:

- `r1` is a `Rect`, so we invoke the `sameShape` method defined in `Rect`.
- This method checks whether `that` (which is `s1`) is an instance of `Square`. Since `Square` is a subclass of `Rect`, this `instanceof` test *returns `true`*. So the method then casts `that` to a `Rect` (which is perfectly fine), and invokes `sameRect`.
- The `sameRect` method compares all the fields of `this` (which is `r1`) to all the fields of `that` (which is `s1`), and since they all match, it returns `true`.

Somehow, our `sameShape` operation is no longer symmetric. The problem is that `instanceof` is too lenient: it returns true when the provided value is an instance of the given class *or of any of its subclasses*. A `Square` is an `instanceof Rect`, but a `Rect` is not an `instanceof Square`, and this asymmetry is now apparent in the behavior of our `sameShape` implementation.

To work around this, we can define a set of methods on the `IShape` interface to tell us when a given shape is precisely a `Circle` or a `Rect` or a `Square`:

```
interface IShape {
  boolean sameShape(IShape that);
  // is this shape a Circle?
  boolean isCircle();
  // is this shape a Rect?
  boolean isRect();
  // is this shape a Square?
  boolean isSquare();
}
```

```
// in Circle
public boolean isCircle() { return true;  }
public boolean isRect()   { return false; }
public boolean isSquare() { return false; }
```

```
// in Rect
public boolean isCircle() { return false; }
public boolean isRect()   { return true;  }
public boolean isSquare() { return false; }
```

```
// in Square
public boolean isCircle() { return false; }
public boolean isRect()   { return false; }
public boolean isSquare() { return true;  }
```

Now if we use these methods instead of `instanceof`, we'll get better results:

```
// in Rect
public boolean sameShape(IShape that) {
  if (that.isRect()) {
    return this.sameRect((Rect)that);
  }
  else {
    return false;
  }
}
```

```
// in Square
public boolean sameShape(IShape that) {
  if (that.isSquare()) {
    return this.sameSquare((Square)that);
```

```
    }
    else {
      return false;
    }
  }
```

> ***Do Now!***
>
> Try making these changes, and confirm that our bad test above is now better.

```
// Comparing a Square with a Rect of the same size
t.checkExpect(s1.sameShape(r1), false)  // Good
t.checkExpect(r1.sameShape(s1), false)  // Good
```

> ***Do Now!***
>
> But `r1` and `s1` both describe rectangles with width and height of 5, at position (3,4). Why are they not equal?

> ***Do Now!***
>
> Why must we have `Square` respond to `isRect` with `false` — surely all `Square`s are `Rect`s?

Remember our original goal for sameness testing: two values are the same if *there is no method we could write, or operation we could use, to distinguish between them.* Now that we have `instanceof`, there *is* a way we can distinguish them:

```
// Distinguishing between a Square and a Rect of the same size
t.checkExpect(s1 instanceof Square, true)
t.checkExpect(r1 instanceof Square, false)
```

If there is a way to distinguish between these values, they must not be the same!

Wrapup

We're still not finished, though: we have these odd `isSomeType` methods which don't have any utility outside of the `sameShape` methods, and we still have casts which (if we make any mistakes) will crash our program with an exception. So how can we implement this functionality *without* casts and *without* oddly-specific helper methods?

> **Exercise**
>
> Try to figure this out. Hint: you may want to look at the `youngerIAT` method from Lecture 7: Accumulator methods, continued, which deals with a similar problem.